Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №2
По дисциплине: «Интеллектуальный анализ данных»
Тема: «Автоэнкодеры»

Выполнил:
Студент 4 курса
Группы ИИ-26
Суховило В. С.
Проверил:
Никитин С. К.

Брест 2026

Цель работы:

Научиться применять автоэнкодеры для осуществления визуализации данных и их анализа.

Общее задание:

1. Реализовать автоэнкодеры с 2 и 3 нейронами в среднем слое и визуализировать результаты.

2. Использовать t-SNE с размерностью 2 и 3 и сравнить несколько значений perplexity.

3. Сравнить результаты с PCA для обоих значений размерности.

Вариант №5. Исходные данные:

UCI Optical Recognition of Handwritten Digits: обучающая часть optdigits.tra (3823 объекта) и тестовая optdigits.tes (1797 объектов); 64 числовых признака, 10 классов.

Реализация и методика эксперимента:

Нормировка интенсивности пикселей диапазона 0…16 к [0,1]. Два полносвязных автоэнкодера (64−32−2(3)−32−64), функция потерь MSE, оптимизатор Adam. PCA — отдельная линейная база сравнения; t-SNE применён в 2D и 3D на фиксированных подвыборках.

Результат программы:

Автоэнкодер	MSE начало	MSE конец
2D	0.15539	0.03902
3D	0.15285	0.03421

Графики и иллюстрации:

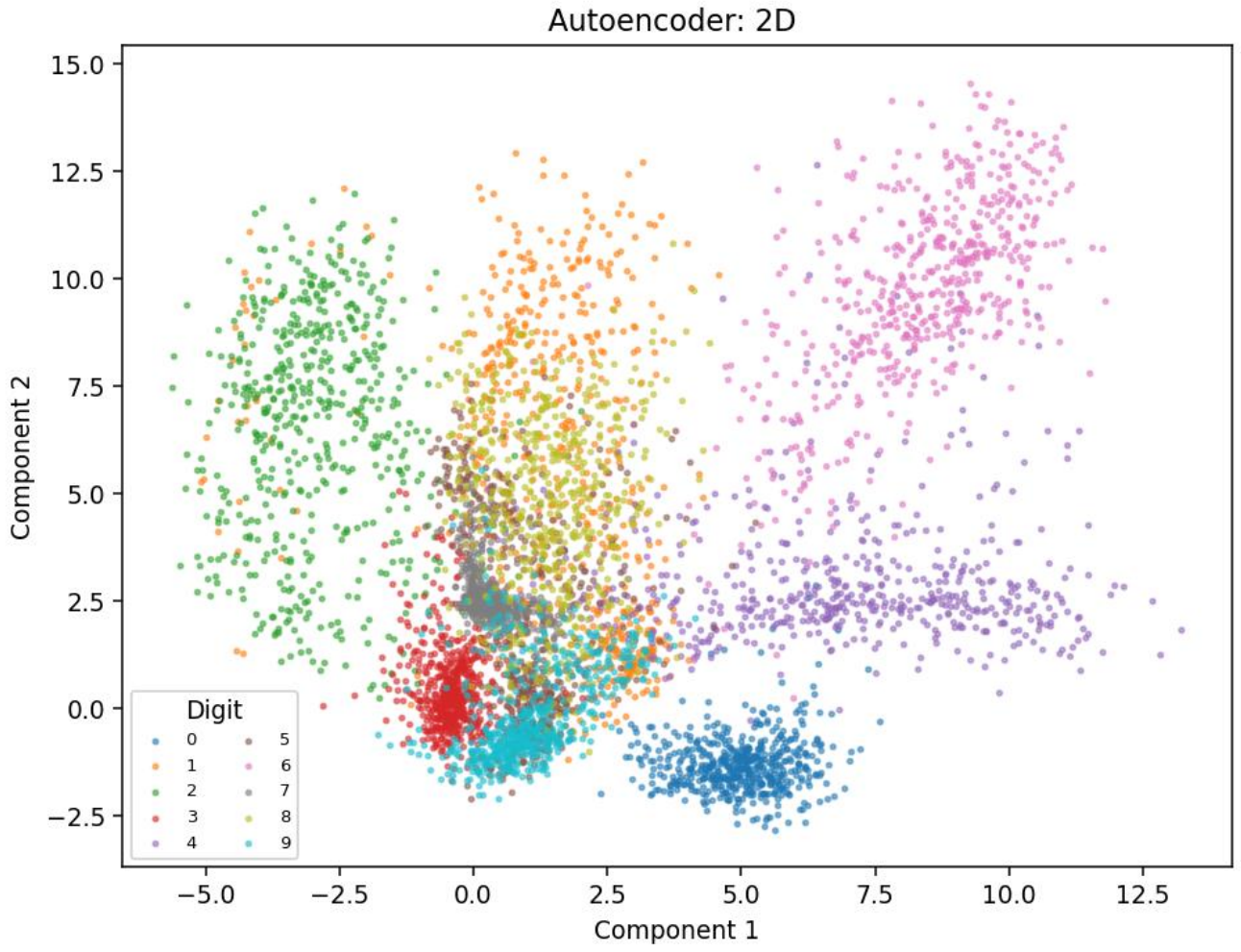


Рисунок. autoencoder 2d

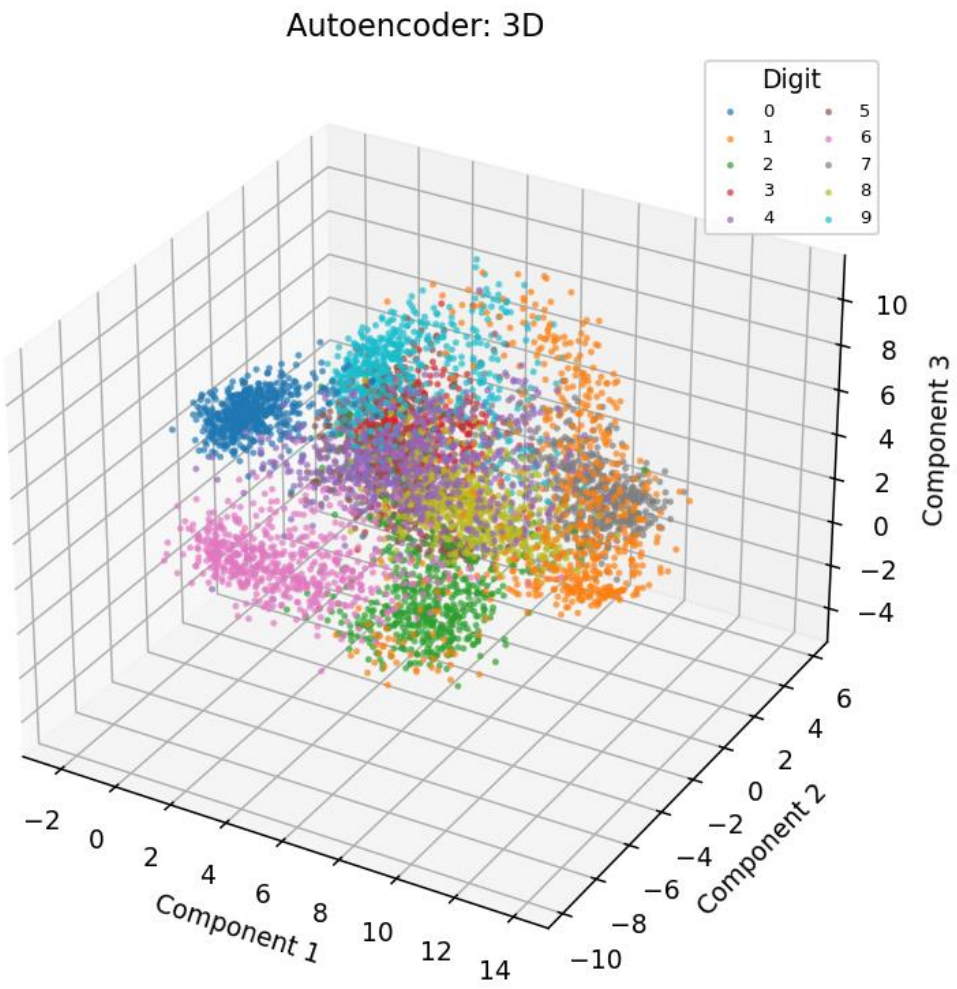


Рисунок. autoencoder 3d

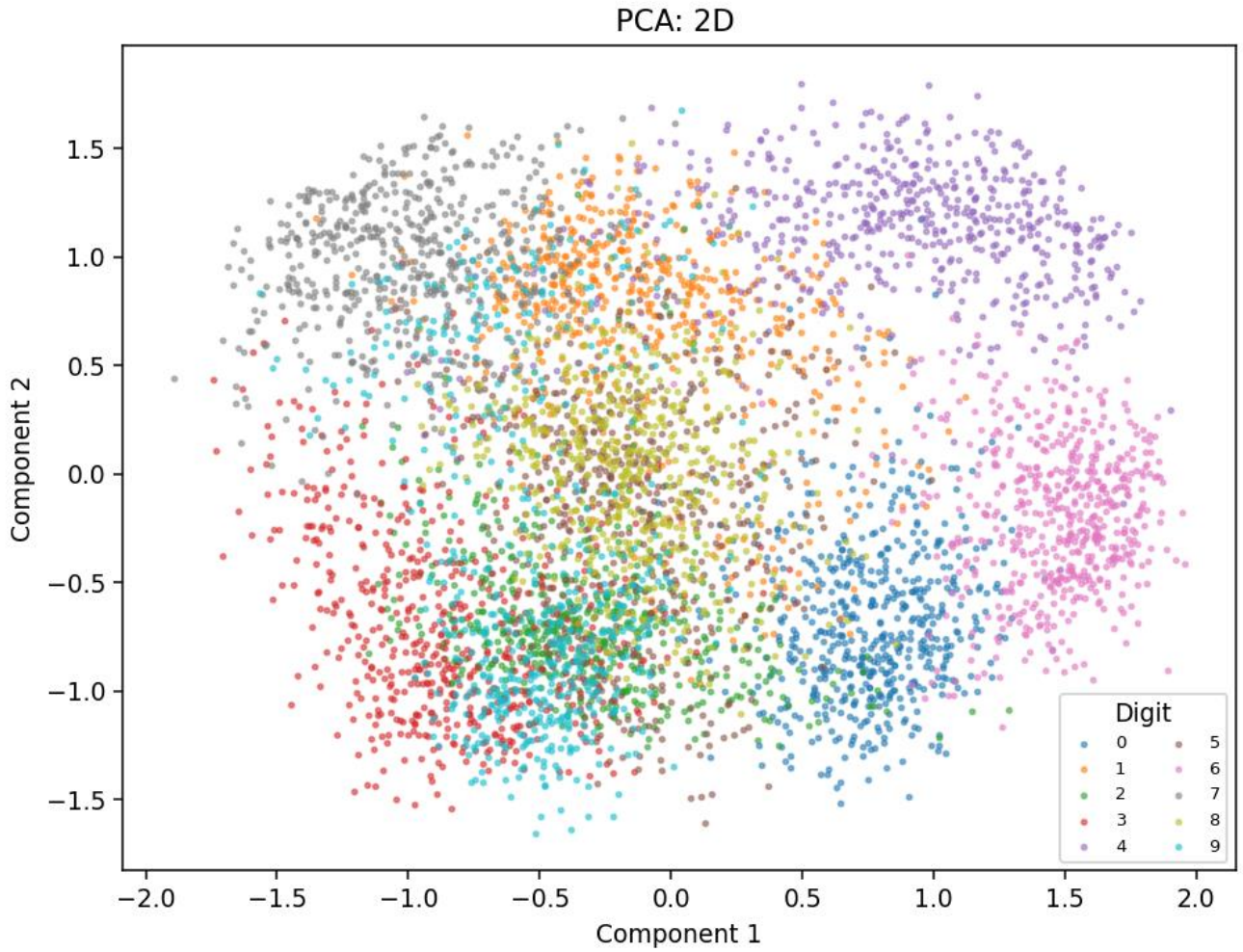


Рисунок. pca 2d

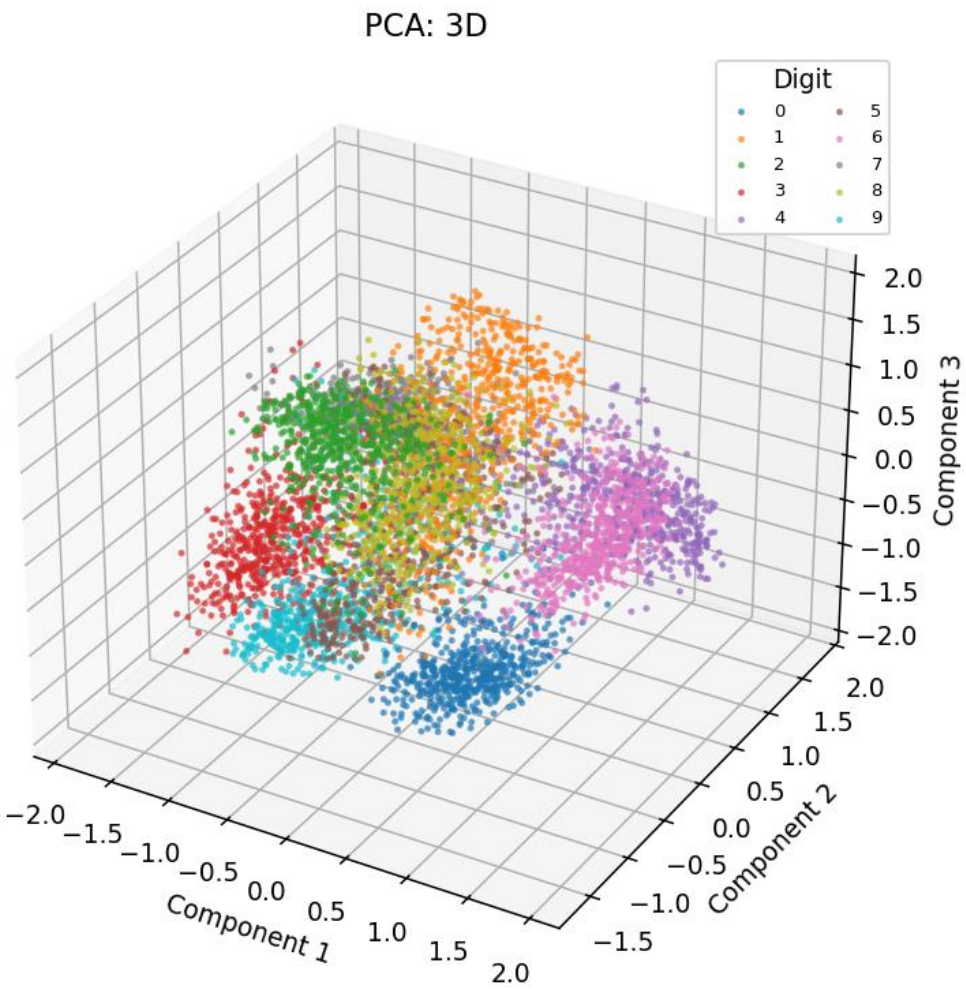


Рисунок. pca 3d

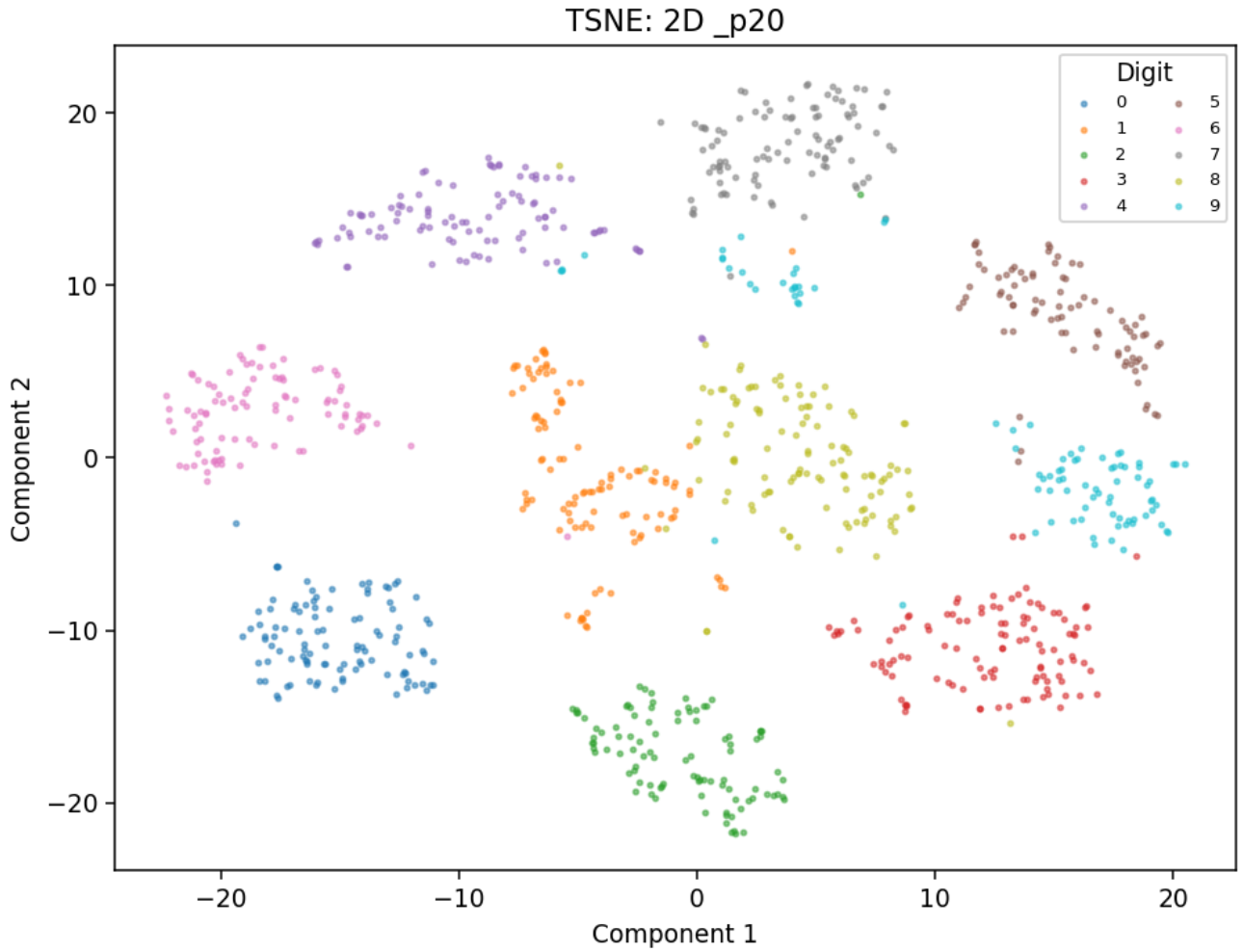


Рисунок. tsne 2d p20

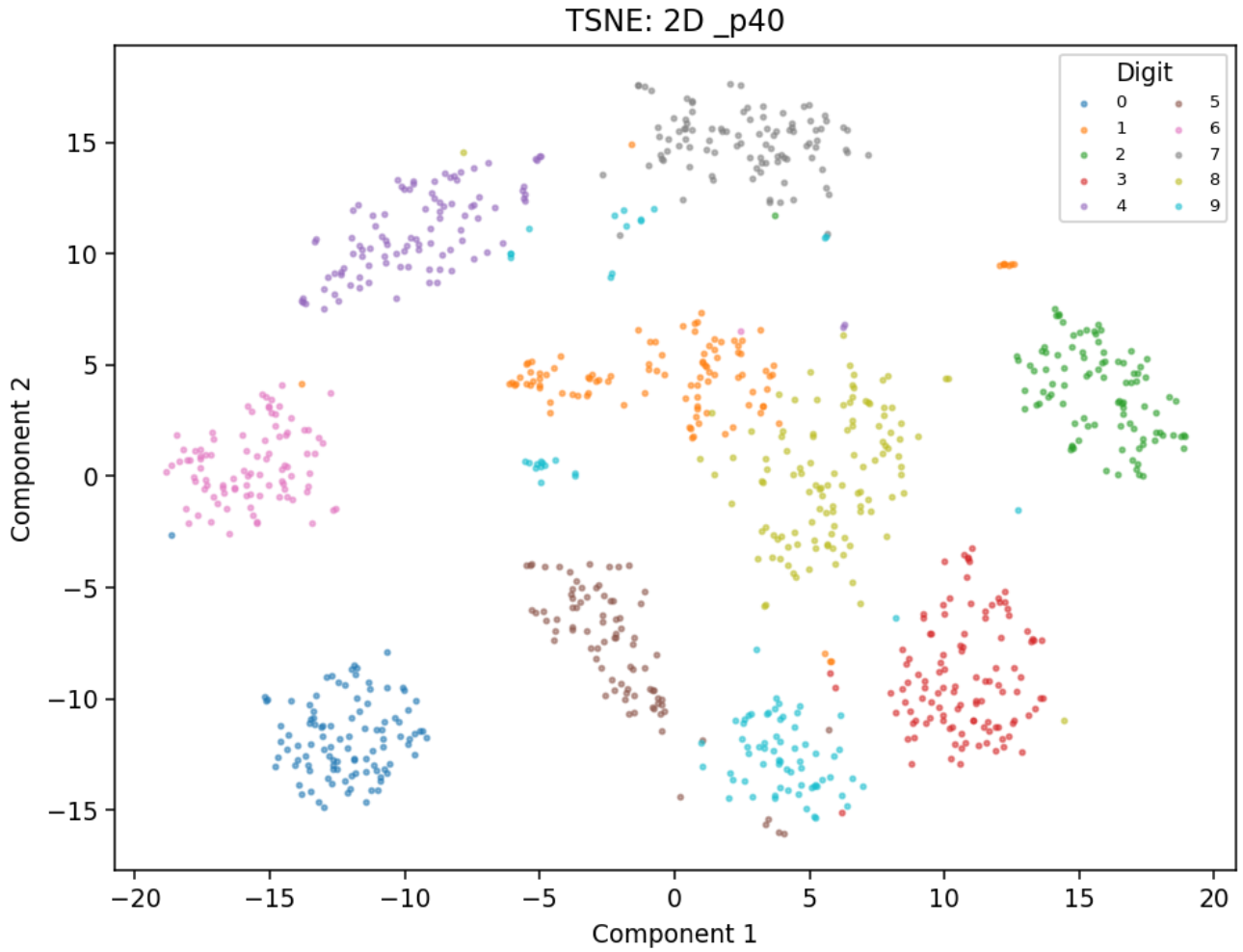


Рисунок. tsne 2d p40

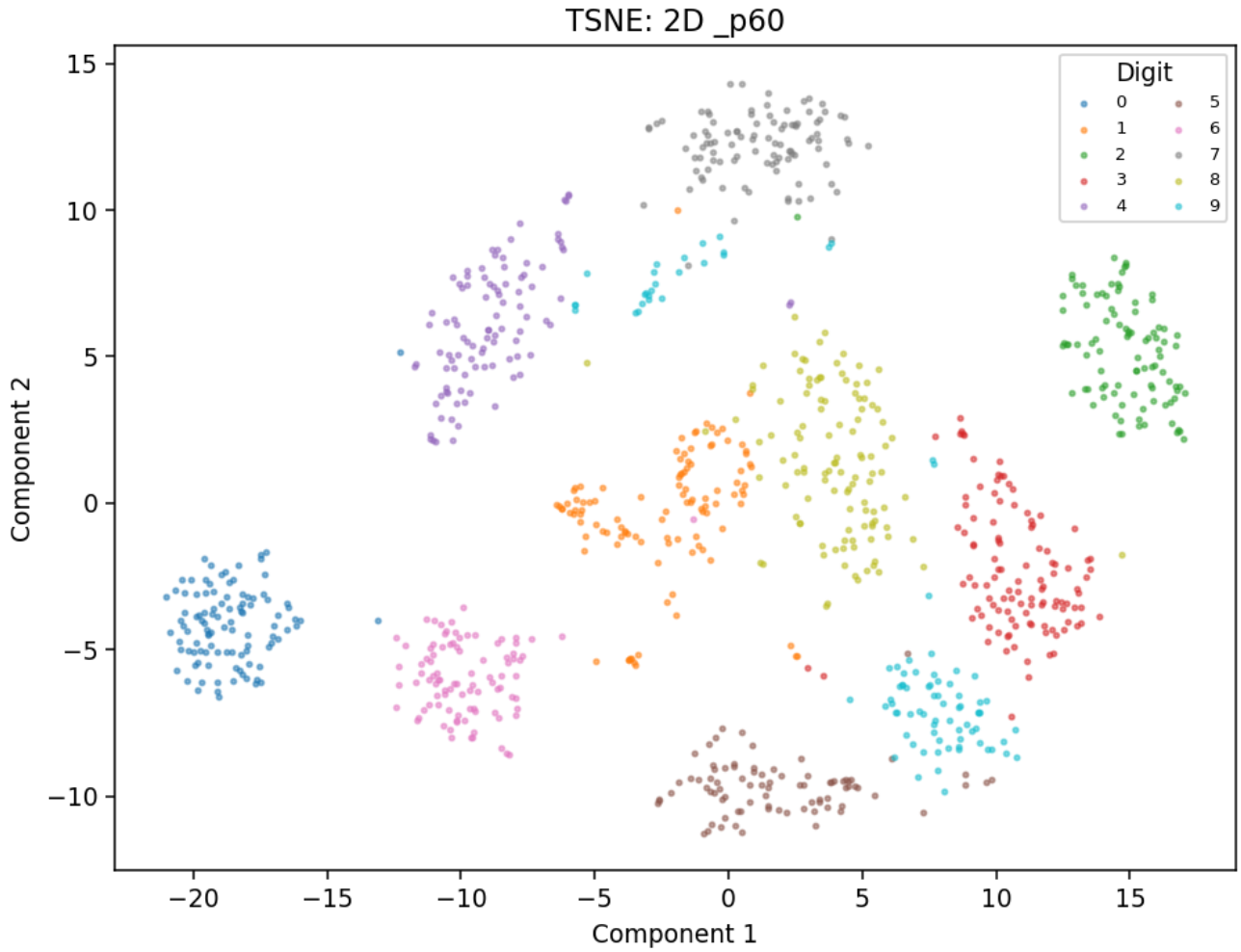


Рисунок. tsne 2d p60

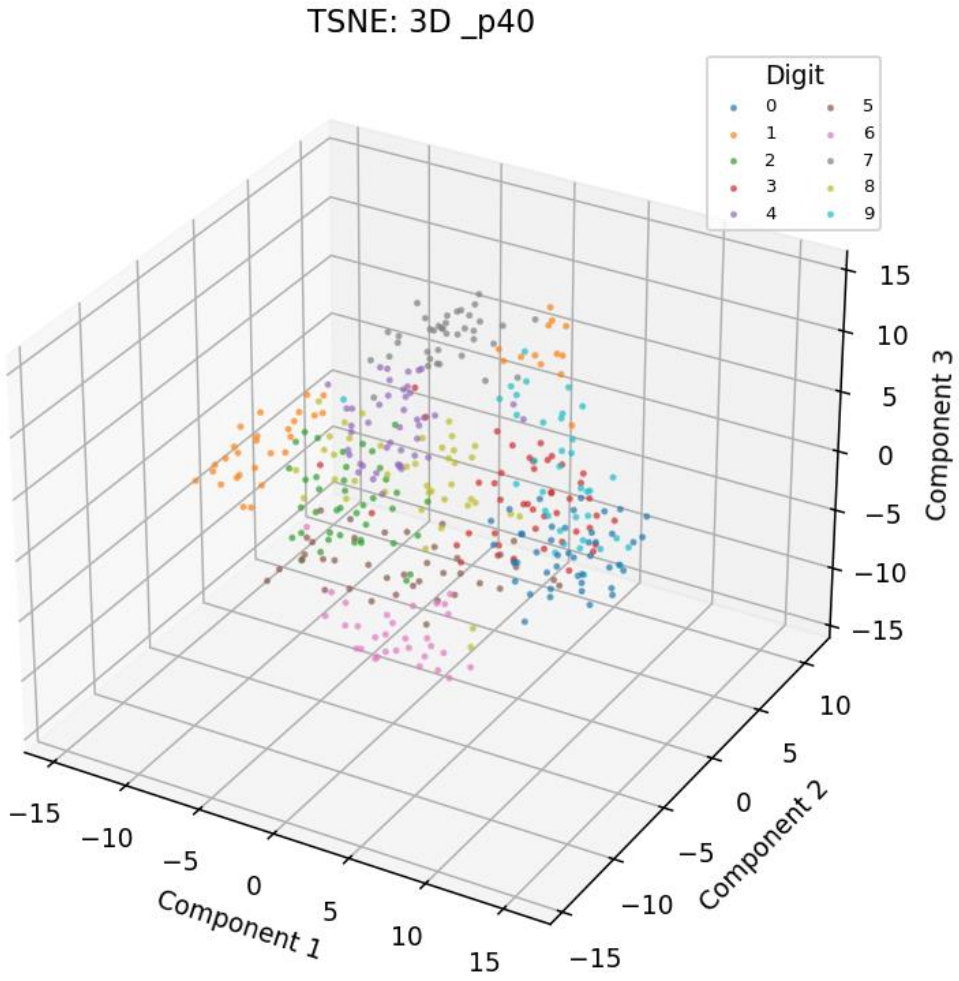


Рисунок. tsne 3d p40

Выводы о проделанной работе:

1. Проанализировано 5620 изображений цифр (64 признака) и 10 классов.

2. Сеть с двумерным bottleneck снизила MSE реконструкции с 0.1554 до 0.0390; трехмерная — с 0.1529 до 0.0342.

3. PCA обеспечивает линейную проекцию, автоэнкодер дает нелинейное представление, а t-SNE ориентирован на сохранение локальной близости объектов. У t-SNE оси не интерпретируются как дисперсии главных компонент.

4. Для t-SNE использованы фиксированные случайные подвыборки: 1000 объектов для 2D и 400 для 3D. Параметры perplexity: 20, 40, 60. Различия визуализаций описывают структуру выборки, а не точность классификатора.

Код программы: ИАД_2.py

```
"""ИАД, ЛР №2, вариант 5: optdigits + Autoencoder, t-SNE, PCA."""
from pathlib import Path
import json,random
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import torch
from torch import nn
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.manifold import TSNE
SEED=42
random.seed(SEED);np.random.seed(SEED);torch.manual_seed(SEED);torch.set_num_threads(2)
ROOT=Path(__file__).resolve().parent;OUT=ROOT/'results';OUT.mkdir(exist_ok=True)
a=pd.read_csv(ROOT/'data/optdigits.tra',header=None)
b=pd.read_csv(ROOT/'data/optdigits.tes',header=None)
df=pd.concat([a,b],ignore_index=True)
X=df.iloc[:,:-1].values.astype(np.float32);y=df.iloc[:,-1].values.astype(int)
# UCI Optical Digits: пиксели 0...16, последний столбец — цифра.
X=X/16.0
class AE(nn.Module):
 def __init__(self,d,latent):

super().__init__();self.enc=nn.Sequential(nn.Linear(d,32),nn.ReLU(),nn.Linear(32,latent));self.dec=nn.Sequential(nn.Linear(lat
ent,32),nn.ReLU(),nn.Linear(32,d),nn.Sigmoid())
 def forward(self,x):return self.dec(self.enc(x))


def train(latent,epochs=55):
 model=AE(X.shape[1],latent);opt=torch.optim.Adam(model.parameters(),lr=.002)
 loader=torch.utils.data.DataLoader(torch.from_numpy(X),batch_size=256,shuffle=True)
 losses=[]
 for ep in range(epochs):
  total=0
  for z in loader:
   opt.zero_grad();loss=nn.functional.mse_loss(model(z),z);loss.backward();opt.step();total+=float(loss)*len(z)
  losses.append(total/len(X))
 with torch.no_grad():out=model.enc(torch.from_numpy(X)).numpy()
 return model,out,losses


def graph(data,kind,k,suffix='',labels=None):
 if labels is None: labels=y
 fig=plt.figure(figsize=(7.2,5.6));ax=fig.add_subplot(111,projection='3d' if k==3 else None)
 for digit in range(10):
  v=data[labels==digit]
  if k==3:ax.scatter(*v.T,s=3,alpha=.55,label=str(digit))
  else:ax.scatter(v[:,0],v[:,1],s=4,alpha=.5,label=str(digit))
 ax.set(xlabel='Component 1',ylabel='Component 2',title=f'{kind}: {k}D {suffix}')
 if k==3:ax.set_zlabel('Component 3')
 ax.legend(title='Digit',ncol=2,fontsize=7,loc='best')
 fig.tight_layout();fig.savefig(OUT/f'{kind.lower().replace(" ","_")}_{k}d{suffix}.png',dpi=135);plt.close(fig)


metrics={'dataset':'UCI optdigits.tra +
optdigits.tes','samples':len(X),'features':X.shape[1],'classes':np.unique(y).tolist(),'autoencoders':{},'tsne':{},'tsne_sample
_size':{'2D':1000,'3D':400}}
for k in (2,3):
 model,z,h=train(k);graph(z,'Autoencoder',k)
 torch.save(model.state_dict(),OUT/f'autoencoder_{k}d.pt')
 metrics['autoencoders'][str(k)]={'epochs':len(h),'initial_mse':h[0],'final_mse':h[-1]}
 plt.figure(figsize=(6,3));plt.plot(h);plt.title(f'Autoencoder {k}D — training
MSE');plt.xlabel('Epoch');plt.ylabel('MSE');plt.tight_layout();plt.savefig(OUT/f'ae_training_{k}d.png',dpi=130);plt.close()
 z=PCA(n_components=k,random_state=SEED).fit_transform(X)
 graph(z,'PCA',k)
 # t-SNE сравниваем при нескольких значениях perplexity.
 for p in (20,40,60):
  # Случайная воспроизводимая подвыборка для вычислительно дорогого t-SNE
  chosen=np.random.default_rng(SEED).choice(len(X),size=(400 if k==3 else 1000),replace=False)
  ts=TSNE(n_components=k,perplexity=p,init='pca',learning_rate='auto',random_state=SEED,max_iter=350,method='barnes_hut' if
k==2 else 'exact')
  q=ts.fit_transform(X[chosen])
  graph(q,'TSNE',k,f'_p{p}',labels=y[chosen])
  metrics['tsne'][f'{k}D_perplexity_{p}']={'kl_divergence':float(ts.kl_divergence_)}
(OUT/'metrics.json').write_text(json.dumps(metrics,indent=2,ensure_ascii=False),encoding='utf8')
print(json.dumps(metrics,ensure_ascii=False,indent=2))
```

Исходный код, данные и графики также приложены к лабораторной работе в директории src/.